AWS Bedrock AgentCore Session Management

This notebook deploys a MCP server, and then run to demo these key concepts, and propose session management best practice to achieve the best throughput.

- AWS Bedrock AgentCore Warm Pool
- Cold Start Latency
- Shared Session vs Unique Session

MCP Server

The MCP Server has two simple tools as shown here. Each tool perform addition, sleep 2 seconds, returns the result, together with SERVER_INSTANCE_ID. The SERVER_INSTANCE_ID is a UUID, which will be used in test to determine if the code runs on the same server instance

```
SERVER_INSTANCE_ID = str(uuid.uuid4())
mcp = FastMCP(host="0.0.0.0", stateless_http=True)

@mcp.tool()
def add_numbers_sync(a: int, b: int) -> str:
    """Add two numbers together (sync)"""
    print(f"Adding numbers (sync): {a} + {b}")
    result = a + b
    time.sleep(2)
    return f"result={result} server={SERVER_INSTANCE_ID}"


@mcp.tool()
async def add_numbers_async(a: int, b: int) -> str:
    """Add two numbers together (async)"""
    print(f"Adding numbers (async): {a} + {b}")
    result = a + b
    await asyncio.sleep(2)
    return f"result={result} server={SERVER_INSTANCE_ID}"
```

Tests

- Invoke a sync MCP tool in parallel, each tool call uses an unique session
- Invoke a sync MCP tool in parallel, all tool calls share the same session
- Invoke an async MCP tool in parallel, each tool call uses an unique session
- Invoke an async MCP tool in parallel, all tool calls share the same session

Based on the testing results, we analysis AgentCore session behaivor, and recommend best approach to build scalable system on AgentCore Runtime

Install Dependencies

Setup

```
Using AWS region: us-west-2
```

Setup AgentCore MCP Server

```
Entrypoint parsed: file=/Users/mingyyzz/dev-local/aws.blog1/src/mcp_server.py, bedrock_agentcore_name=mcp_server
Memory disabled - agent will be stateless
Configuring BedrockAgentCore agent: blog_mcp_math
Memory disabled
Network mode: PUBLIC
📄 Using existing Dockerfile: /Users/mingyyzz/dev-local/aws.blog1/src/Dockerfile
Generated .dockerignore: /Users/mingyyzz/dev-local/aws.blog1/src/.dockerignore
Keeping 'blog_mcp_math' as default agent
Bedrock AgentCore configured: /Users/mingyyzz/dev-local/aws.blog1/src/.bedrock_agentcore.yaml
🚀 Launching Bedrock AgentCore (cloud mode - RECOMMENDED)...
   • Deploy Python code directly to runtime
   • No Docker required (DEFAULT behavior)
   • Production-ready deployment

💡 Deployment options:
   • runtime.launch()                → Cloud (current)
   • runtime.launch(local=True)      → Local development
Memory disabled - skipping memory creation
Starting CodeBuild ARM64 deployment for agent 'blog_mcp_math' to account 980413094772 (us-west-2)
Generated image tag: 20260322-172750-132
Setting up AWS resources (ECR repository, execution roles)...
Getting or creating ECR repository for agent: blog_mcp_math
ECR repository available: 980413094772.dkr.ecr.us-west-2.amazonaws.com/bedrock-agentcore-blog_mcp_math
Getting or creating execution role for agent: blog_mcp_math
Using AWS region: us-west-2, account ID: 980413094772
Role name: AmazonBedrockAgentCoreSDKRuntime-us-west-2-622e8ad068
✅ Reusing existing ECR repository: 980413094772.dkr.ecr.us-west-2.amazonaws.com/bedrock-agentcore-blog_mcp_math
```

```
✅ Reusing existing execution role: arn:aws:iam::980413094772:role/AmazonBedrockAgentCoreSDKRuntime-us-west-2-622e8ad068
Execution role available: arn:aws:iam::980413094772:role/AmazonBedrockAgentCoreSDKRuntime-us-west-2-622e8ad068
Preparing CodeBuild project and uploading source...
Getting or creating CodeBuild execution role for agent: blog_mcp_math
Role name: AmazonBedrockAgentCoreSDKCodeBuild-us-west-2-622e8ad068
Reusing existing CodeBuild execution role: arn:aws:iam::980413094772:role/AmazonBedrockAgentCoreSDKCodeBuild-us-west-2-622e8ad068
Using dockerignore.template with 47 patterns for zip filtering
Uploaded source to S3: blog_mcp_math/source.zip
Updated CodeBuild project: bedrock-agentcore-blog_mcp_math-builder
Starting CodeBuild build (this may take several minutes)...
Starting CodeBuild monitoring...
🔄 QUEUED started (total: 0s)
✅ QUEUED completed in 1.1s
🔄 PROVISIONING started (total: 1s)
✅ PROVISIONING completed in 7.4s
🔄 DOWNLOAD_SOURCE started (total: 8s)
✅ DOWNLOAD_SOURCE completed in 1.1s
🔄 BUILD started (total: 10s)
✅ BUILD completed in 11.6s
🔄 POST_BUILD started (total: 21s)
✅ POST_BUILD completed in 6.3s
🔄 COMPLETED started (total: 27s)
✅ COMPLETED completed in 1.0s
🎉 CodeBuild completed successfully in 0m 28s
CodeBuild completed successfully
CodeBuild project configuration saved
Deploying to Bedrock AgentCore...
Agent created/updated: arn:aws:bedrock-agentcore:us-west-2:980413094772:runtime/blog_mcp_math-Z057Lr3Pdg
Observability is enabled, configuring observability components...
CloudWatch Logs resource policy already configured
X-Ray trace destination already configured
X-Ray indexing rule already configured
Transaction Search already fully configured
ObservabilityDeliveryManager initialized for region: us-west-2, account: 980413094772
✅ Logs auto-created by AWS for runtime/blog_mcp_math-Z057Lr3Pdg
✅ Traces delivery enabled for runtime/blog_mcp_math-Z057Lr3Pdg
Observability enabled for runtime/blog_mcp_math-Z057Lr3Pdg - logs: True, traces: True
✅ X-Ray traces delivery enabled for agent blog_mcp_math-Z057Lr3Pdg
🔍 GenAI Observability Dashboard:
   https://console.aws.amazon.com/cloudwatch/home?region=us-west-2#gen-ai-observability/agent-core
Polling for endpoint to be ready...
Agent endpoint: arn:aws:bedrock-agentcore:us-west-2:980413094772:runtime/blog_mcp_math-Z057Lr3Pdg/runtime-endpoint/DEFAULT
Deployment completed successfully - Agent: arn:aws:bedrock-agentcore:us-west-2:980413094772:runtime/blog_mcp_math-Z057Lr3Pdg
Built with CodeBuild: bedrock-agentcore-blog_mcp_math-builder:1654d998-c654-4faa-bba2-a8890e0b5437
Deployed to cloud: arn:aws:bedrock-agentcore:us-west-2:980413094772:runtime/blog_mcp_math-Z057Lr3Pdg
ECR image: 980413094772.dkr.ecr.us-west-2.amazonaws.com/bedrock-agentcore-blog_mcp_math:20260322-172750-132
🔍 Agent logs available at:
   /aws/bedrock-agentcore/runtimes/blog_mcp_math-Z057Lr3Pdg-DEFAULT --log-stream-name-prefix "2026/03/22/\[runtime-logs]"
   /aws/bedrock-agentcore/runtimes/blog_mcp_math-Z057Lr3Pdg-DEFAULT --log-stream-names "otel-rt-logs"
💡 Tail logs with: aws logs tail /aws/bedrock-agentcore/runtimes/blog_mcp_math-Z057Lr3Pdg-DEFAULT --log-stream-name-prefix "2026/03/22/\[runtime-logs]" --follow
💡 Or view recent logs: aws logs tail /aws/bedrock-agentcore/runtimes/blog_mcp_math-Z057Lr3Pdg-DEFAULT --log-stream-name-prefix "2026/03/22/\[runtime-logs]" --sin
Launch completed. Agent ARN: arn:aws:bedrock-agentcore:us-west-2:980413094772:runtime/blog_mcp_math-Z057Lr3Pdg
```

```
{'Version': 5,
 'Tier': 'Standard',
 'ResponseMetadata': {'RequestId': '28ebb7b9-231e-44b7-b7d6-832be500c7db',
  'HTTPStatusCode': 200,
  'HTTPHeaders': {'server': 'Server',
   'date': 'Sun, 22 Mar 2026 17:28:22 GMT',
   'content-type': 'application/x-amz-json-1.1',
   'content-length': '31',
   'connection': 'keep-alive',
   'x-amzn-requestid': '28ebb7b9-231e-44b7-b7d6-832be500c7db',
   'cache-control': 'no-store'},
  'RetryAttempts': 0}}
```

Sync MCP Tool Parallel Tests

AWS Bedrock AgentCore maintains a **Warm Pool of 10 instances** for each deployed MCP server. These pre-initialized instances are ready to serve requests immediately — with no cold start latency. The warm pool is replenished approximately once per minute as instances are consumed.

The following tests demonstrate how the warm pool affects latency for **sync** tools under different concurrency and session configurations.

Call the sync tool with 5 parallel threads, each call uses unique session — No Cold Start

All 5 requests are served by **different instances from the warm pool** (5 unique server IDs). Since the warm pool has 10 instances available, 5 concurrent requests are fully absorbed — every call completes in ~3s (the 2s `sleep` + ~1s overhead), with **zero cold start latency**.

```
add_numbers_sync — 5 concurrent calls, unique sessions
  add_numbers_sync #0   start=10:20:30, end=10:20:33, duration=  3.00s, server=2a55175b-187a-49e3-b640-625f168af94e
  add_numbers_sync #1   start=10:20:30, end=10:20:33, duration=  2.98s, server=a523cfb8-5836-4ba4-98ad-4449b0027176
  add_numbers_sync #2   start=10:20:30, end=10:20:33, duration=  3.02s, server=2d8b828f-2293-46ea-b1a3-1cfc89cc962c
  add_numbers_sync #4   start=10:20:30, end=10:20:33, duration=  3.01s, server=b5be6d96-238f-4dbb-808c-453fd9498c29
  add_numbers_sync #3   start=10:20:30, end=10:20:34, duration=  3.27s, server=13c32c71-5c7a-4747-8549-98e4feba9b06
  total wall time: 3.32s  avg per call: 3.06s
  unique server instances: 5
```

Call the sync tool with 15 parallel threads, unique sessions — Cold Start Beyond Warm Pool

With 15 concurrent requests exceeding the warm pool size of 10:

- **First 10 invocations (~3s each):** Served immediately by warm pool instances — no cold start
- **Remaining 5 invocations (~10-12s each):** Must wait for new instances to cold-start, adding ~7-9s of overhead

This confirms the warm pool holds exactly **10 instances**. Requests beyond the pool capacity trigger cold starts, which take significantly longer as AgentCore must provision and initialize new runtime instances.

```
add_numbers_sync — 15 concurrent calls, unique sessions
  add_numbers_sync #4   start=10:35:10, end=10:35:13, duration=  3.05s, server=5375d49f-3e23-4fbf-8632-39013d01b579
  add_numbers_sync #8   start=10:35:10, end=10:35:13, duration=  3.03s, server=754ec58f-df86-42ae-a859-f2ef833add10
  add_numbers_sync #3   start=10:35:10, end=10:35:13, duration=  3.08s, server=cbb8682a-debd-4f4b-aabb-a88106c74594
  add_numbers_sync #0   start=10:35:10, end=10:35:13, duration=  3.17s, server=af7f0ed9-fa64-4ecc-b9df-bfcc7a1f75cc
  add_numbers_sync #9   start=10:35:10, end=10:35:13, duration=  3.03s, server=da8adb0c-e7e9-465c-8976-2214b8f83502
  add_numbers_sync #2   start=10:35:10, end=10:35:13, duration=  3.11s, server=e58b0778-64d7-4cfa-ba7a-482c5bf25402
  add_numbers_sync #13  start=10:35:10, end=10:35:13, duration=  3.01s, server=91d4ed92-355d-45f5-9e30-f265ae05de2f
  add_numbers_sync #6   start=10:35:10, end=10:35:13, duration=  3.11s, server=fb886245-eb37-43d7-85fe-eeb7c3b31965
  add_numbers_sync #7   start=10:35:10, end=10:35:13, duration=  3.13s, server=6e5b3671-9451-4acc-9783-e4932b149a2c
  add_numbers_sync #11  start=10:35:10, end=10:35:13, duration=  3.10s, server=930bf772-b379-4bb0-8cd1-eefeb323a6a5
  add_numbers_sync #10  start=10:35:10, end=10:35:20, duration= 10.09s, server=accbb431-cc8e-4a9f-9e34-47a9be369122
  add_numbers_sync #5   start=10:35:10, end=10:35:20, duration= 10.14s, server=8f10ade6-038c-4fb4-bcc1-aaf44325141e
  add_numbers_sync #12  start=10:35:10, end=10:35:22, duration= 12.11s, server=66d42f94-dc19-4930-a35c-036a227eb981
  add_numbers_sync #14  start=10:35:10, end=10:35:22, duration= 12.13s, server=aecbfa55-10d5-4e9a-80bb-32c07d146d41
  add_numbers_sync #1   start=10:35:10, end=10:35:22, duration= 12.28s, server=646531be-c2e0-46ac-8854-6bcda7c9ae2f
  total wall time: 12.34s  avg per call: 5.84s
  unique server instances: 15
```

Call the sync tool with 15 parallel threads, shared session — Sequential Bottleneck

All 15 invocations are routed to the **same instance** (1 unique server ID) because they share a session. Since `add_numbers_sync` uses `time.sleep(2)` — which blocks the thread — requests are processed **sequentially**. Each call must wait for the previous one to finish:

- Total wall time: **~31s** (≈ 15 × 2s sleep + overhead)
- Average per call: **~28.6s** (most calls wait in queue)

Shared session with sync tools creates a severe bottleneck. The server cannot leverage the warm pool for parallelism since session affinity pins all requests to one instance.

```
add_numbers_sync — 15 concurrent calls, shared Mcp-Session-Id=a1f57bca-fbd5-4614-9faa-f5966b193f16
  add_numbers_sync #11  start=10:41:22, end=10:41:42, duration= 20.89s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #12  start=10:41:22, end=10:41:42, duration= 20.88s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #10  start=10:41:21, end=10:41:43, duration= 21.04s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #6   start=10:41:21, end=10:41:45, duration= 23.09s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #4   start=10:41:21, end=10:41:53, duration= 31.14s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #9   start=10:41:21, end=10:41:53, duration= 31.10s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #1   start=10:41:21, end=10:41:53, duration= 31.18s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #5   start=10:41:21, end=10:41:53, duration= 31.14s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #8   start=10:41:21, end=10:41:53, duration= 31.12s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #14  start=10:41:22, end=10:41:53, duration= 31.06s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #0   start=10:41:21, end=10:41:53, duration= 31.25s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #2   start=10:41:21, end=10:41:53, duration= 31.29s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #7   start=10:41:21, end=10:41:53, duration= 31.25s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #13  start=10:41:22, end=10:41:53, duration= 31.20s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  add_numbers_sync #3   start=10:41:21, end=10:41:53, duration= 31.32s, server=1ca5deec-5203-4c57-b267-ef48a3faac4c
  total wall time: 31.40s  avg per call: 28.60s
  unique server instances: 1
```

Async MCP Tool Parallel Tests

The same warm pool of **10 instances** applies to async tools. However, async tools using `asyncio.sleep()` yield the event loop, allowing a single instance to handle **multiple concurrent requests**. This fundamentally changes the performance profile compared to sync tools.

Call the async tool with 5 parallel threads, unique sessions — No Cold Start

Same as the sync case: all 5 requests are served by warm pool instances with **no cold start**. Each call completes in ~3s. The warm pool comfortably handles 5 concurrent requests.

```
add_numbers_async — 5 concurrent calls, unique sessions
  add_numbers_async #4   start=10:46:37, end=10:46:40, duration=  2.88s, server=1e23d15d-354f-4ba8-af7e-be5d8784312a
  add_numbers_async #3   start=10:46:37, end=10:46:40, duration=  2.94s, server=2f9b327f-6137-4ed7-9788-d6a4be911e71
  add_numbers_async #2   start=10:46:37, end=10:46:40, duration=  2.98s, server=d9886c4f-8d57-48f5-9460-cea8f70ca9c3
  add_numbers_async #1   start=10:46:37, end=10:46:40, duration=  3.00s, server=46b2fe6f-808e-4991-a312-a7469c3982e3
  add_numbers_async #0   start=10:46:37, end=10:46:40, duration=  3.09s, server=12c094d8-0f09-4c4b-9a5c-9061380c88d8
  total wall time: 3.09s  avg per call: 2.98s
  unique server instances: 5
```

Call the async tool with 15 parallel threads, unique sessions — Cold Start Beyond Warm Pool

With 15 concurrent requests far exceeding the warm pool:

- **First 10 invocations (~3s):** Served by warm pool — no cold start
- **Next ~5 invocations (~10s):** Cold-started instances, ~7s overhead

All 15 calls use different server instances (15 unique IDs). The warm pool absorbs the first 10 instantly; the rest incur cold start penalties. Despite this, total wall time is **~12s** because all instances run in parallel once started.

```
add_numbers_async — 15 concurrent calls, unique sessions
  add_numbers_async #5   start=17:22:06, end=17:22:10, duration=  3.26s, server=ab991a55-6a2d-453c-8cac-ed481af437a7
  add_numbers_async #2   start=17:22:06, end=17:22:10, duration=  3.33s, server=fbfc0fa5-71f1-480b-a997-68d134892646
  add_numbers_async #0   start=17:22:06, end=17:22:10, duration=  3.44s, server=61ddf2f4-f567-40d8-b57b-389512c5c738
  add_numbers_async #9   start=17:22:07, end=17:22:10, duration=  3.26s, server=5e698bf7-3f0b-443f-bca3-23994498829b
  add_numbers_async #8   start=17:22:07, end=17:22:10, duration=  3.28s, server=28c7f20d-2db7-4733-b2ce-6ce552ce1dcd
  add_numbers_async #4   start=17:22:06, end=17:22:10, duration=  3.33s, server=5dd36ec6-a725-4f70-a380-83d57ebce34f
  add_numbers_async #3   start=17:22:06, end=17:22:10, duration=  3.36s, server=1904f8ed-a443-47eb-9cce-030e15361389
  add_numbers_async #12  start=17:22:07, end=17:22:10, duration=  3.26s, server=ac19b7e7-81e1-4dff-8c84-a060aafe36d7
  add_numbers_async #6   start=17:22:06, end=17:22:10, duration=  3.36s, server=ccb20245-6984-4c0a-a4a0-770da026d48f
  add_numbers_async #14  start=17:22:07, end=17:22:10, duration=  3.38s, server=758cd7f6-f711-46ac-9057-cc276d6065e9
  add_numbers_async #10  start=17:22:07, end=17:22:17, duration= 10.08s, server=8e51d278-e6ff-4715-9a8d-551d335d0a0e
  add_numbers_async #7   start=17:22:07, end=17:22:17, duration= 10.23s, server=915edb7f-1d5a-4e7d-82b0-d9d5f9ad7132
  add_numbers_async #1   start=17:22:06, end=17:22:17, duration= 10.31s, server=c7ff5705-56cf-4c28-a392-a6ade203c421
  add_numbers_async #13  start=17:22:07, end=17:22:19, duration= 12.07s, server=1f3c588e-da86-4c6a-9dc0-2e50b28175db
  add_numbers_async #11  start=17:22:07, end=17:22:19, duration= 12.11s, server=7e0d9aa2-7d3b-4287-8dd5-1aabfa73a8bf
  total wall time: 12.31s  avg per call: 5.87s
  unique server instances: 15
```

Call the async tool with 15 parallel threads, shared session — Best Performance

This is the **optimal configuration**. All 15 requests are routed to a **single instance** (shared session), but because the tool is async, the event loop handles all 15 calls **concurrently**:

- **All 15 invocations complete in ~3s** — same as a single call
- Total wall time: **~3.3s** (vs 31s for sync shared session, vs 12s for async unique session)
- Average per call: **~3.05s**

Why is this the fastest? With async + shared session:

1. **No cold start** — only 1 warm pool instance is needed (we have 10 available)
2. **True concurrency** — `asyncio.sleep` yields the event loop, so all 15 calls execute their 2s wait simultaneously
3. **No warm pool exhaustion** — a single instance serves all traffic, leaving the remaining 9 warm pool instances available for other workloads

```
add_numbers_async — 15 concurrent calls, shared Mcp-Session-Id=409f874c-5e6c-47a6-b171-9ca2994a7dc8
  add_numbers_async #1   start=12:03:19, end=12:03:22, duration=  2.99s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #11  start=12:03:19, end=12:03:22, duration=  2.95s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #9   start=12:03:19, end=12:03:22, duration=  2.97s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #6   start=12:03:19, end=12:03:22, duration=  3.00s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #12  start=12:03:19, end=12:03:22, duration=  2.97s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #14  start=12:03:19, end=12:03:22, duration=  2.95s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #10  start=12:03:19, end=12:03:22, duration=  3.01s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #0   start=12:03:19, end=12:03:22, duration=  3.14s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #3   start=12:03:19, end=12:03:22, duration=  3.09s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #8   start=12:03:19, end=12:03:22, duration=  3.07s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #7   start=12:03:19, end=12:03:22, duration=  3.08s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #2   start=12:03:19, end=12:03:22, duration=  3.14s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #13  start=12:03:19, end=12:03:22, duration=  3.06s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #4   start=12:03:19, end=12:03:22, duration=  3.14s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  add_numbers_async #5   start=12:03:19, end=12:03:22, duration=  3.19s, server=0d22d2f0-2aa0-4a13-a3d2-d275cb561898
  total wall time: 3.26s  avg per call: 3.05s
  unique server instances: 1
```

Analysis & Recommendations

Key Findings

Test	Tool	Session	Concurrency	Wall Time	Avg/Call	Cold Starts
1	sync	unique	5	~3s	~3.1s	0
2	sync	unique	15	~12s	~5.8s	5
3	sync	shared	15	~31s	~28.6s	0
4	async	unique	5	~3s	~3.0s	0
5	async	unique	15	~12s	~5.8s	5
6	async	shared	15	**~3s**	**~3.1s**	**0**

1. Warm Pool: 10 Instances, Replenished Per Minute

AgentCore maintains a warm pool of **10 pre-initialized instances** per MCP server deployment. Evidence:

- Tests with ≤10 concurrent unique-session calls show **zero cold start** — all complete in ~3s
- Tests with >10 concurrent unique-session calls show exactly **10 fast calls** followed by slower cold-started ones
- The warm pool replenishes approximately **once per minute**, so back-to-back burst tests may see reduced warm pool availability

2. Warm Pool Eliminates Cold Start Latency

Warm pool instances serve requests with near-zero initialization overhead:

- **Warm pool latency:** ~3s (2s tool sleep + ~1s network/framework overhead)
- **Cold start latency:** ~10-12s (additional ~7-9s for instance provisioning)

Cold start penalty is **3-4x** the baseline latency. For latency-sensitive workloads, keeping concurrency within the warm pool size (≤10) avoids cold starts entirely.

3. Async + Shared Session = Best Throughput

The async shared-session test (Test 6) achieves the best performance across all configurations:

- **15 concurrent calls in ~3s** — the same time as a single call
- Uses only **1 instance**, leaving the rest of the warm pool free
- Scales with the event loop's concurrency capacity rather than instance count

This works because `async` tools yield the event loop during I/O waits (`await asyncio.sleep`), enabling true concurrent execution within a single process. In contrast, sync tools with `time.sleep` block the thread, forcing sequential execution on a shared session.

4. Security Tradeoffs of Shared Sessions

While async + shared session delivers the best performance, it introduces important security considerations:

Risks of shared sessions:

- **Session isolation:** All requests on a shared session are routed to the same server instance and share the same process memory. If the MCP tool maintains any in-memory state (caches, variables, context), one caller's data could leak to another
- **Blast radius:** A single malicious or malformed request could crash the shared instance, affecting all users on that session
- **Audit & attribution:** With a shared session ID, it becomes harder to trace which caller triggered a specific action in server-side logs
- **Privilege confusion:** If the MCP server performs authorization based on session context (e.g., user identity, permissions), shared sessions could lead to privilege escalation where one caller inherits another's access level

When shared sessions are appropriate:

- Internal/trusted workloads where all callers have the same privilege level
- Stateless tools that don't maintain per-request context beyond the call
- Read-only operations with no side effects
- Single-tenant deployments where all requests originate from the same user or service

When to use unique sessions:

- Multi-tenant environments where callers must be isolated
- Tools that perform writes, mutations, or actions with side effects
- Workloads requiring per-caller audit trails

- Scenarios where session-level state (auth tokens, user context) must not cross boundaries

Recommendation: For production workloads, use **async tools with unique sessions** as the default, accepting the warm pool limit as your concurrency ceiling. Reserve shared sessions for trusted, stateless, high-throughput internal pipelines where the performance gain justifies the reduced isolation.

Cleanup (Optional)